



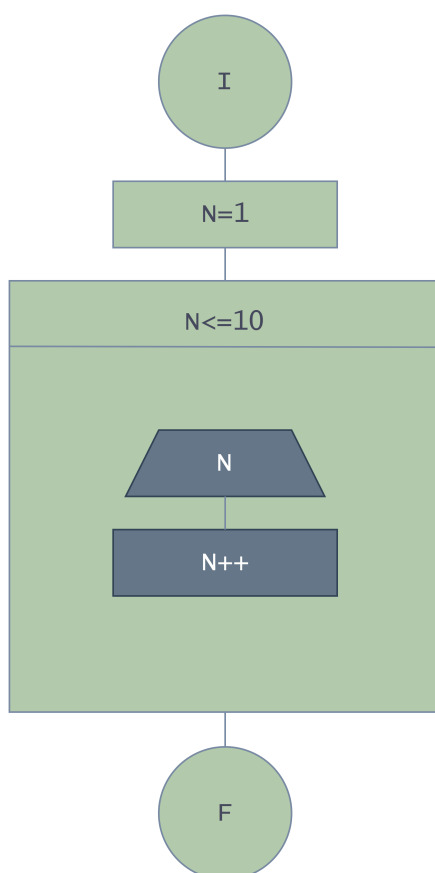
Guía de Ejercicios (Ciclo WHILE) – Resueltos.

Leer atentamente y resolver en diagrama de flujo.

Requerido: Videos E1 a E11 (especialmente este último) de la serie para Aprender Programación Desde Cero.

¡No olvides que no alcanza con leer la problemática una sola vez! Dedícale todas las lecturas y el análisis que sea requerido. Repasá con los videos! Consultá las dudas. Y desde ya, tenés que usar el ciclo WHILE en esta guía.

1. Hacer un programa para mostrar los números del 1 al 10. No se debe realizar ningún pedido de datos. USANDO WHILE.



Este ejercicio lo resolvimos en la Guía pasada de Ciclo FOR, en este caso lo aplicaremos, pero utilizando un Ciclo While, como sabemos este ciclo es inexacto y va a iterar dependiendo la condición con la que fue configurado.

Bien, para este programa empezamos creando la variable **N** y le asignamos un 1.

Seguido a esto debemos configurar nuestro ciclo, para este caso le decimos que mientras el valor de **N** sea menor o igual a 10 siga iterando.

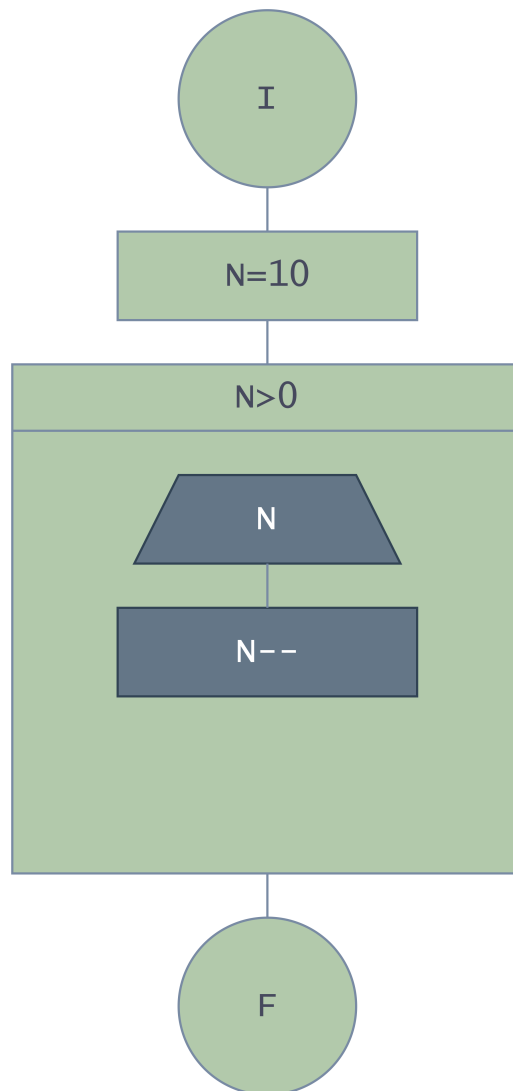
Dentro del Ciclo imprimimos en pantalla el valor de **N**, posterior le incrementamos su valor.

Nuestro programa en la primera vuelta nos va a mostrar un 1 porque se lo asignamos fuera del Ciclo, después al incrementar su valor, en la segunda vuelta nos va a mostrar un 2 y así sucesivamente hasta que llegue a 10.

Para que muestre hasta el 10 inclusive hay 2 formas de configurar el Ciclo, una podría ser **N < 11** y la otra como lo tenemos configurado en este diagrama **N <= 10**.

Una vez que termino de Ciclar, sale y termina nuestro programa.

2. Hacer un programa para mostrar los números del 10 al 1. No se debe realizar ningún pedido de datos. USANDO WHILE.



En este ejercicio aplicamos una lógica similar a la anterior que jugamos con la condición del Ciclo. Empezamos creando la variable **N** y le asignamos un 10.

Seguido, creamos la condición de nuestro ciclo, en este caso **N > 0**, mientras el valor de **N** sea mayor a 0 seguí dando vueltas.

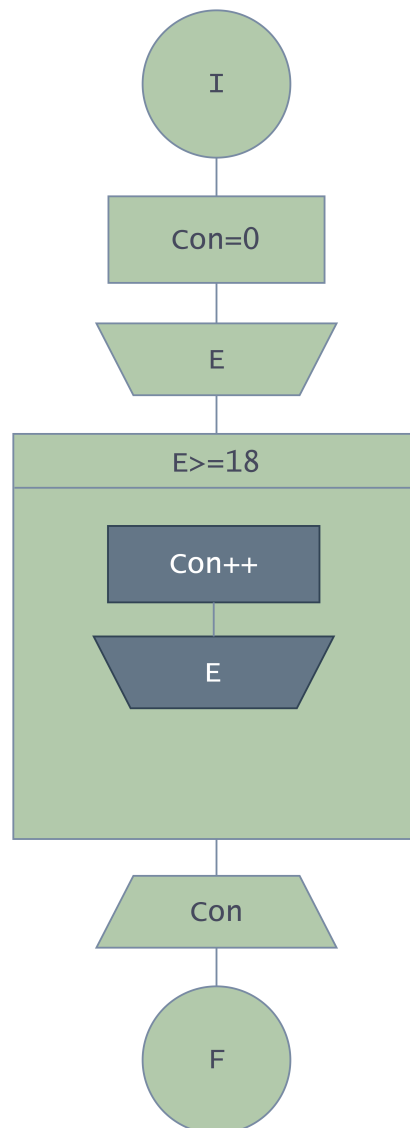
Dentro del Ciclo mostramos el valor de **N** en pantalla y le disminuimos su valor en 1 **N--**.

Como nos pide que mostremos los números del 10 al 1 de forma decreciente, empezamos asignando un 10 a **N** fuera del Ciclo, para que cuando entre al mismo, en la primera vuelta nos imprima un 10. Luego decrementa su valor y en la segunda vuelta nos va a mostrar un 9 y así sucesivamente hasta llegar al 1. Cuando **N** valga 0 se va a chocar con nuestra condición y como 0 no es mayor 0, sale del Ciclo.

Hay 2 formas para configurar la condición del Ciclo. Una es la forma planteada en este ejemplo y otra podría ser **N >= 1**.

Cuando termine de iterar sale del ciclo y termina nuestro programa.

3. Hacer un programa que solicite la edad de un grupo de personas. El programa deberá pedir edades hasta que se ingrese una edad menor a 18 años. Deberá mostrar por pantalla cuántas personas mayores se registraron.



Para este programa, teniendo en cuenta la consigna, ya empezamos a armar nuestra condición.

“... Deberá pedir edades hasta que se ingrese una edad menor a 18 años”

¿Que nos está diciendo esto? Básicamente que nuestro ciclo de vueltas hasta que se ingrese una edad menor a 18. Con esto en mente, ya empezamos a armar nuestra condición.

Por otro lado, nos dice: “... Mostrar **cuántas** personas mayores se registraron”; si analizamos esto, ya nos damos cuenta de que vamos a necesitar un contador.

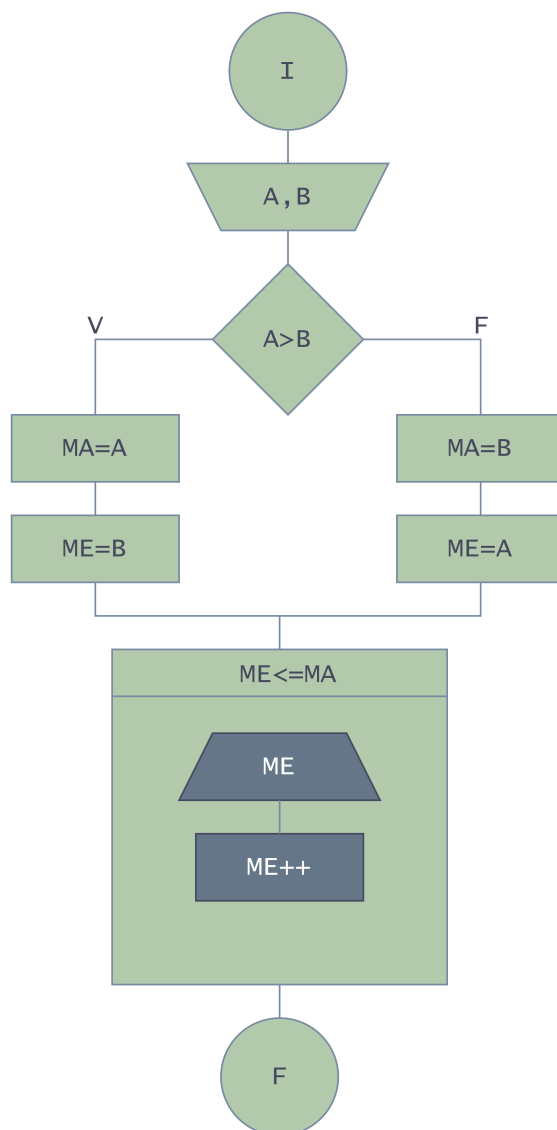
Empezamos creando nuestro contador y lo iniciamos en 0. **CON = 0** y pedimos el ingreso de una edad **E**.

Seguimos con la condición de nuestro ciclo **E >= 18** y dentro del ciclo incrementamos el contador y volvemos a pedir la edad **E**.

Explicación de su funcionamiento: Inicia nuestro programa, el contador arranca en 0, pedimos una edad. ¿Y por qué pedimos la edad fuera del ciclo? Bueno, porque necesitamos ese dato para poder armar la condición. Si no cargamos ninguna edad, cuando se tope con el ciclo, no va a entrar porque no tenemos nada dentro de Edad, y nuestra condición dice que para que itere el ciclo, Edad tiene que ser mayor o igual a 18.

Una vez dentro del While, se incrementa el contador++ y pedimos una edad nuevamente: si es mayor o igual a 18 va a seguir iterando tantas; veces como cumpla la condición. Cuando no se cumpla la condición, sale del ciclo, mostramos el conteo que obtuvimos en pantalla **CON** y termina nuestro programa.

4. Hacer un programa que solicite dos números y luego muestre los números entre el menor y el mayor de ellos. Acordate: usando While.



Este programa nos pide que mostremos los números entre el menor y el mayor ingresado. El usuario va a ingresar dos números, por ejemplo 3 y 7, y nuestro programa deberá mostrar los números 3,4,5,6,7. La consigna no especifica si debemos resolver mostrando inclusive los números ingresados, así que podemos mostrarlos o no.

Comenzamos pidiendo el ingreso de 2 números que se alojaran en las variables **A** y **B**.

Luego, para diferenciar el mayor y el menor de estos, hacemos un **IF (A > B)** donde estamos preguntando si el valor de **A** es mayor que el valor de **B**.

Si **A** es mayor, va a tomar el camino del verdadero y guardamos **A** como el mayor "**MA = A**" y **B** como el menor "**ME = B**".

Si **B** es mayor, va a tomar el camino del falso y guardamos **B** como el mayor "**MA = B**" y **A** como el menor "**ME = A**".

En este punto ya tenemos diferenciado nuestro menor número ingresado y también el mayor. Nos queda mostrar en pantalla los números que están entre estos. Para eso vamos a crear un ciclo While. En la condición le decimos que mientras **ME** sea menor o igual a **MA** siga iterando: "**ME <= MA**".

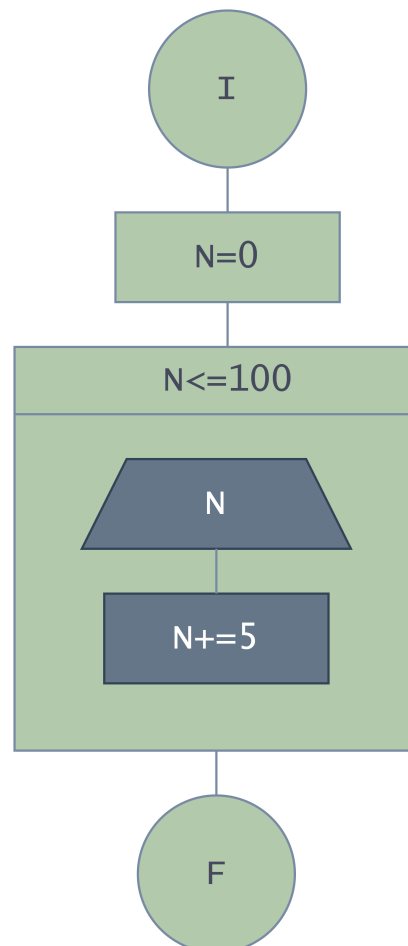
Dentro del ciclo mostramos en pantalla el valor de **ME** e incrementamos su valor.

Con el ejemplo de números planteados nos quedaría en la primera vuelta del While un 3, se incrementa su valor y en la segunda vuelta nos muestra un 4 y así sucesivamente hasta mostrar el 7 inclusive, ya que en la condición le dijimos que dé vueltas (que itere) mientras **ME** sea menor o igual a **MA**.

En nuestra resolución se van a mostrar inclusive los números ingresados.

¿Cómo sería para que muestre sólo los números entre excluyendo los 2 ingresados? Te lo dejo de desafío.

5. Hacer un programa que muestre los números del 1 al 100 de 5 en 5. Ejemplo: 0, 5, 10, 15, 20... 100. Usando While.



Para este programa vamos a tomar el ejemplo del enunciado como referencia y vamos a mostrar los números del 0 al 100.

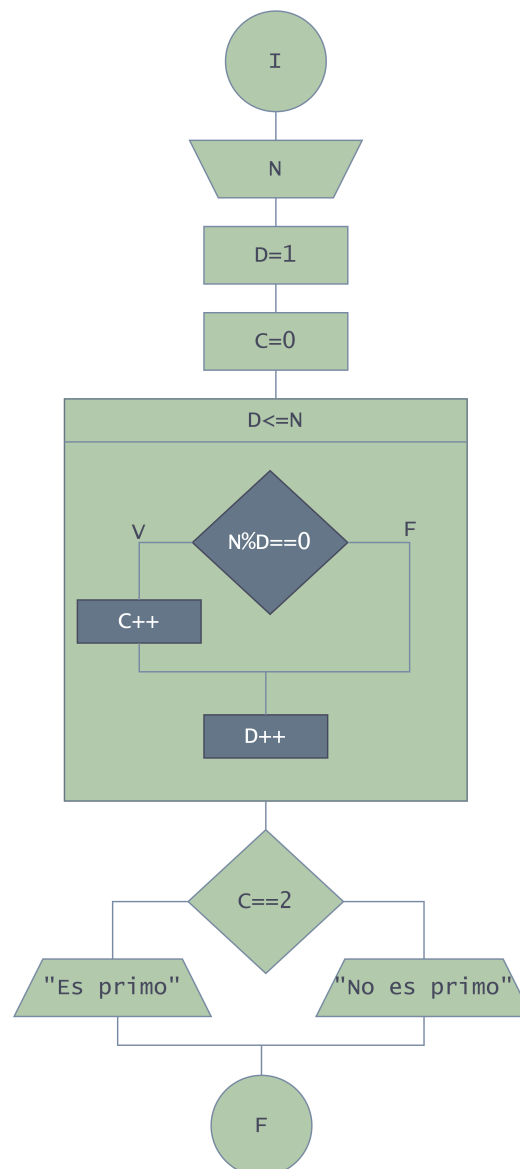
Empezamos creando una variable **N** y le asignamos un 0. Armamos nuestro ciclo y en la condición le decimos que mientras el valor de **N** sea menor o igual a 100, siga iterando: "**N <= 100**".

Dentro del ciclo, mostramos en pantalla el valor de **N** y luego aumentamos su valor en +5: "**N += 5**".

En la primera vuelta nos va a mostrar un 0, se incrementa su valor y en la segunda vuelta nos imprime un 5 y así sucesivamente hasta que **N** valga 100. En este punto **N** es igual a 100 y como configuramos nuestro ciclo para que itere mientras sea menor o igual a 100, lo va a incluir. Lo muestra en pantalla, se incrementa una vez más quedando nuestra variable en 105 y finalmente, cuando se topa con la condición del ciclo, sale, porque su valor superó el 100. Con esto quedaría terminado nuestro programa.

6. Hacer un programa que solicite UN número y luego calcule y emita un cartel aclaratorio si el mismo es primo o no es primo.

Nota: usando While. Ya lo hicimos con FOR, ahora con While.



Para realizar este ejercicio vamos a pedir el ingreso de un número y vamos a analizar si es primo o no. Esto ya lo hicimos en ejercicios anteriores, lo que cambia en esta ocasión es que vamos a utilizar un ciclo `While`.

Empezamos pidiendo el ingreso de un número N , creamos la variable D y le asignamos un 1. Esta variable nos va a servir para ir evaluando el resto obtenido de dividir N y D .

Creamos la variable C y le asignamos un 0. Esta variable nos va a servir para ir contando cuántas veces obtuvimos resto 0 de la división entre N y D . Hasta acá es igual como lo veníamos haciendo antes.

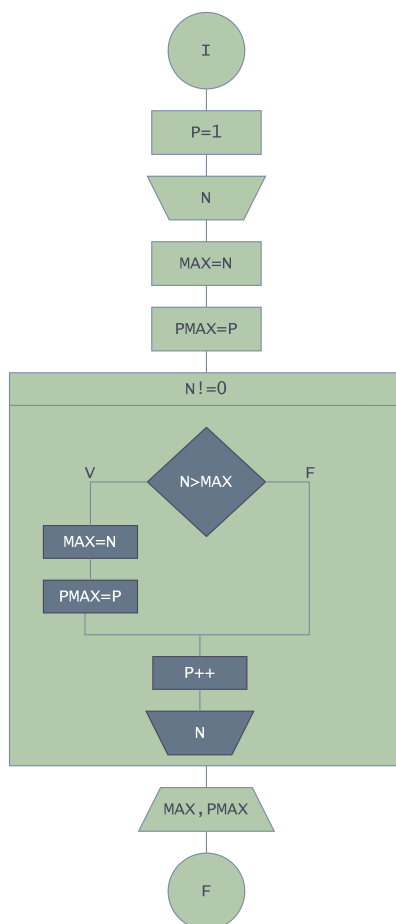
En la creación del ciclo `While` configuramos la condición " $D \leq N$ ", lo que nos dice que mientras el valor de D sea menor o igual al número ingresado, siga iterando. Con esta condición nos aseguramos de que se ingrese un número positivo, ya que D inicia en 1 y, por otro lado, si se ingresa un 1 también va a entrar al Ciclo, ya que va a ser igual a D .

Dentro del ciclo, creamos un condicional ($N \% D == 0$) Si el resto de dividir N con D da 0, tomará el camino del verdadero e incrementamos el valor de C con la instrucción $C++$. En caso de que dé falso, no hacemos nada.

Y así va a iterar nuestro programa tantas veces como el número ingresado. Al salir del ciclo, creamos otro IF donde vamos a determinar si el número ingresado fue primo o no: ($C == 2$). Si el valor de C es igual a 2 es porque el número ingresado fue primo, entonces toma este camino y nos muestra en pantalla "Es primo".

En el caso de que C no fuese igual a 2 es porque nuestro número no fue primo y mostramos en pantalla "No es primo". Con esto tendríamos nuestro programa terminado para evaluar si un número es primo o no utilizando ciclo `While`.

7. Hacer un programa que solicite una lista de números que corta cuando se ingresa un cero y luego mostrar por pantalla el máximo de ellos y la posición en la que fue ingresado.



Para este programa tenemos que tener en cuenta cómo vamos a ir contando las vueltas y como se guarda la posición del número máximo.

Empezamos creando la variable **P** iniciada en 1. Esta variable es la encargada de llevar el conteo, la iniciamos en 1 porque antes de entrar al ciclo ya tenemos el primer ingreso de un número.

A continuación pedimos el ingreso de un número que se alojará en **N**.

Creamos la variable **MAX** y le asignamos el número ingresado: **MAX = N**. Vamos a tomar este primer valor como el máximo, ya que es el único número ingresado hasta el momento.

Para guardar la posición en que se ingresó el Máximo, creamos la variable **PMAX** y le asignamos la posición actual que será contabilizada con la posición **P**. Nos quedaría **PMAX = P**.

Como la condición nos pide que se ingresen números hasta que se ingrese un 0, la condición la configuramos de la siguiente forma **N != 0**. Esto nos dice que mientras el valor de **N** sea distinto a 0, siga iterando.

Dentro del ciclo creamos un IF **que** nos va a servir para ir buscando el mayor ingresado (**N > MAX**) Esto nos dice si el valor de **N** es mayor al valor de **MAX** vamos al verdadero.

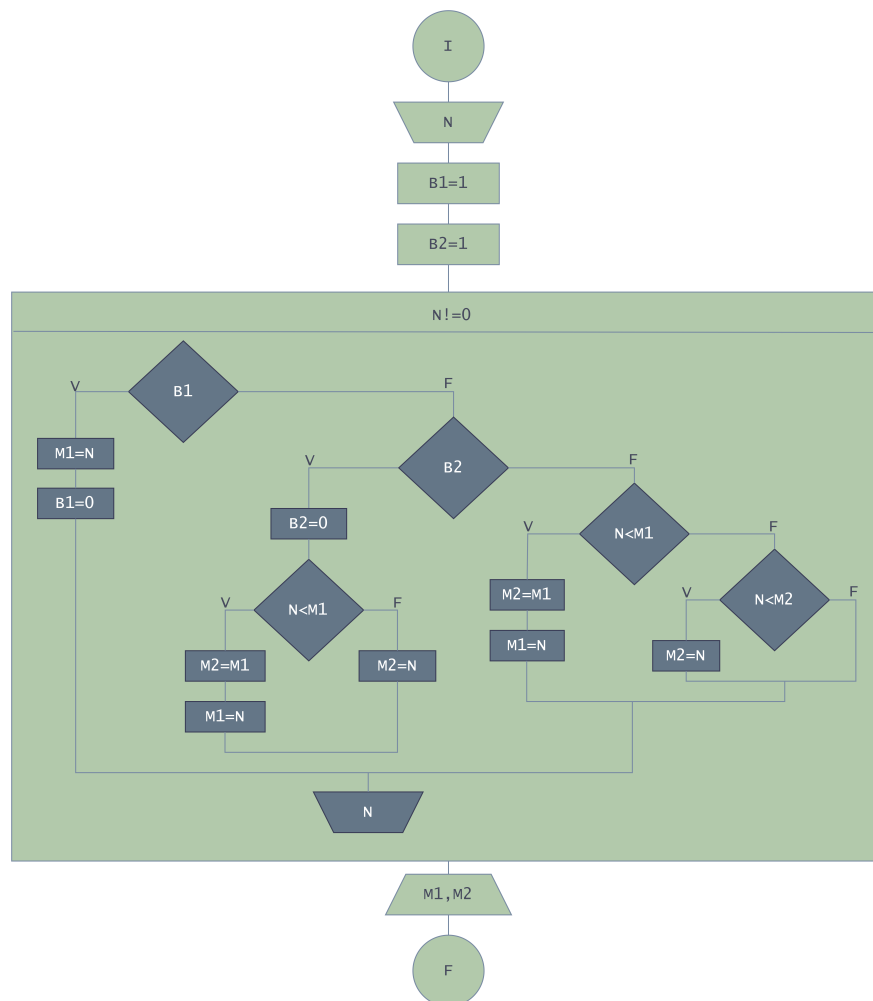
Si nos da verdadero es porque el número que se ingresó fue mayor, entonces reemplazamos el valor que tenemos en **MAX** por el número ingresado: **MAX = N**. También debemos guardar la posición en que se ingresó, para esto pisamos la posición del máximo por la posición actual: **PMAX = P**.

Vaya al verdadero o vaya al falso, al salir del IF incrementamos el valor de posición **P++**. Recordemos que esta variable debe incrementar en todas las vueltas porque nos va marcando la posición actual, o sea, en qué vuelta del ciclo estamos, una especie de "índice"; como la "x" del For, pero aquí las la inventamos.

Luego volvemos a pedir otro número y se repite todo el proceso. Tener en cuenta que, en la primera vuelta del ciclo, cuando se tope con el IF va a ir directo al falso porque en la primera vuelta **N** va a ser igual al **MAX**, esto no perjudica en nada ya que estamos incrementando la posición al final del ciclo y volvemos a pedir otro número. En la vuelta 2 del ciclo la posición va a estar sincronizada.

Para terminar, mostramos en pantalla el valor de **MAX** y **PMAX**.

8. Hacer un programa que solicite una lista de números que corta cuando se ingresa un cero y luego mostrar por pantalla el menor y el segundo menor.



Para este programa empezamos pidiendo el ingreso de un número **N**, luego creamos 2 variables del tipo BOOL o Bandera, y les asignamos un 1 para que inicializarlas en verdadero.

En la configuración del ciclo: **N != 0**. Esto nos dice que va a iterar hasta que se ingrese un 0 o, visto de otro modo, “mientras N sea distinto de cero”.

Primer IF **B1**: este primer IF nos sirve para guardar el primer ingreso como el menor. Una vez que ingresa, cambiamos el valor de la bandera a 0 para que en la siguiente vuelta de falso.

Segundo IF **B2**: este IF nos va a servir para guardar el segundo número menor ingresado. Una vez que ingresa por el camino del verdadero, cambiamos su valor a falso: **B=0**.

Luego creamos otro IF **N < M1**, que nos sirve para saber si el número que se ingresó es menor a **M1**. Si da verdadero es porque este número es más chico que el que guardamos en la primera vuelta, entonces tenemos que hacer una especie de enroque: pasamos el valor de **M1** a **M2** y el nuevo número ingresado lo alojamos en **M1**. En el caso que dé falso, simplemente guardamos este número como el segundo menor **M2 = N**.

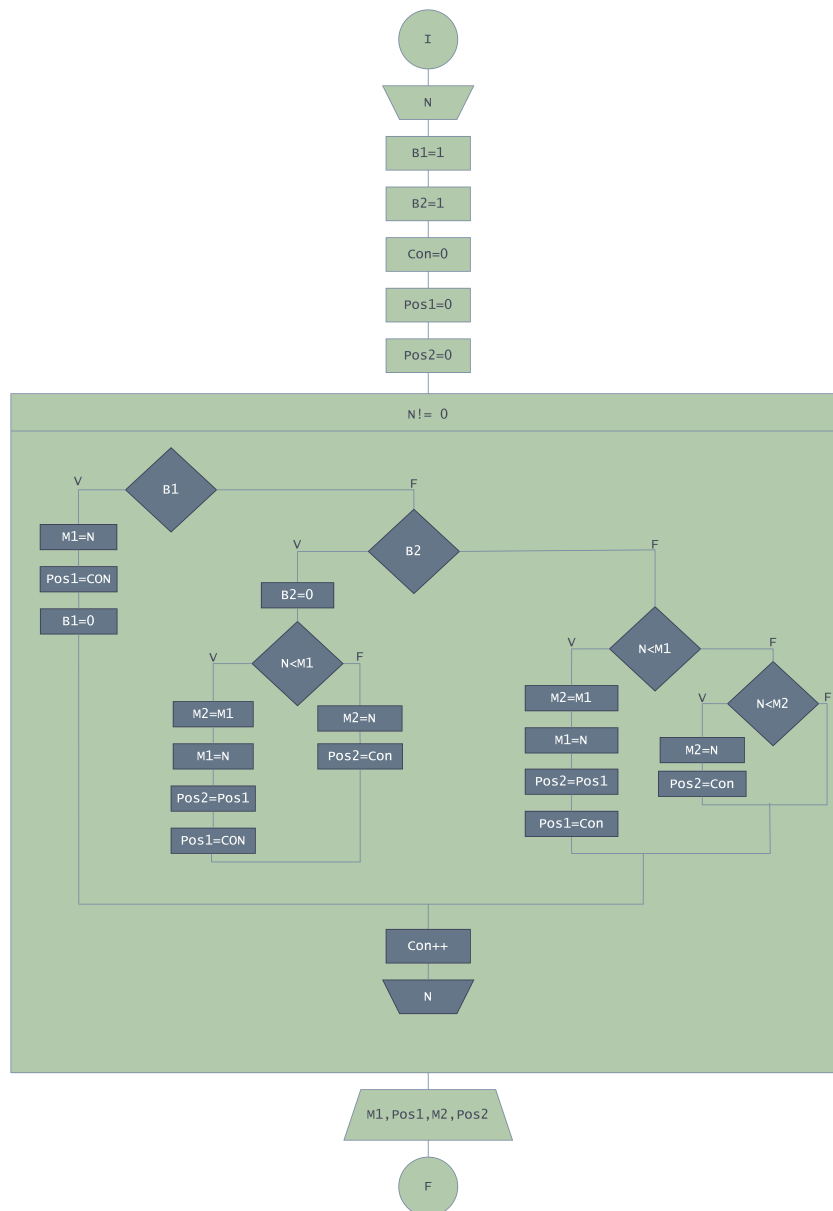
Una vez que ya paso por los 2 primeros IF “**B1 Y B2**”, tendremos nuestro **M1 Y M2** ya cargados. Ambos IF los cambiamos a falso, así que en la tercera vuelta del ciclo va a “saltar” estos 2 IF.

Con **M1 Y M2** cargados solo nos queda ir comparando sus valores con los nuevos ingresos. Para esto creamos un IF (**N < M1**) para determinar si el número que se ingresó es menor a **M1**, si es menor va a ir para el verdadero hacemos de nuevo el enroque **M2 = M1 y M1 = N**.

En el caso que dé falso, tenemos que hacer otro IF **N < M2** para corroborar si es menor a **M2**, porque podría ser que no fuera menor a **M1**, pero sí que sea menor a **M2**, entonces antes de asignarlo a **M2** corroboramos que sea menor, en el caso que de verdadero **M2 = N**. En el caso que de falso no hacemos nada.

Para finalizar, mostramos los valores en pantalla **M1, M2** y terminará nuestro programa.

9. Realizar nuevamente el ejercicio 8, pero ahora debe devolver además la posición en la que fue encontrado cada uno de los mínimos.



Este programa es igual que el anterior con la diferencia que vamos a agregar las variables de posiciones. Las variables que agregamos son:

CON = 0 que nos va a dar el índice o contando las vueltas que da nuestro ciclo.

POS1 = 0 que nos va a servir para guardar la posición en que se ingresó el primer menor.

POS2 = 0 que nos sirve para guardar la posición en que se ingresó el segundo menor.

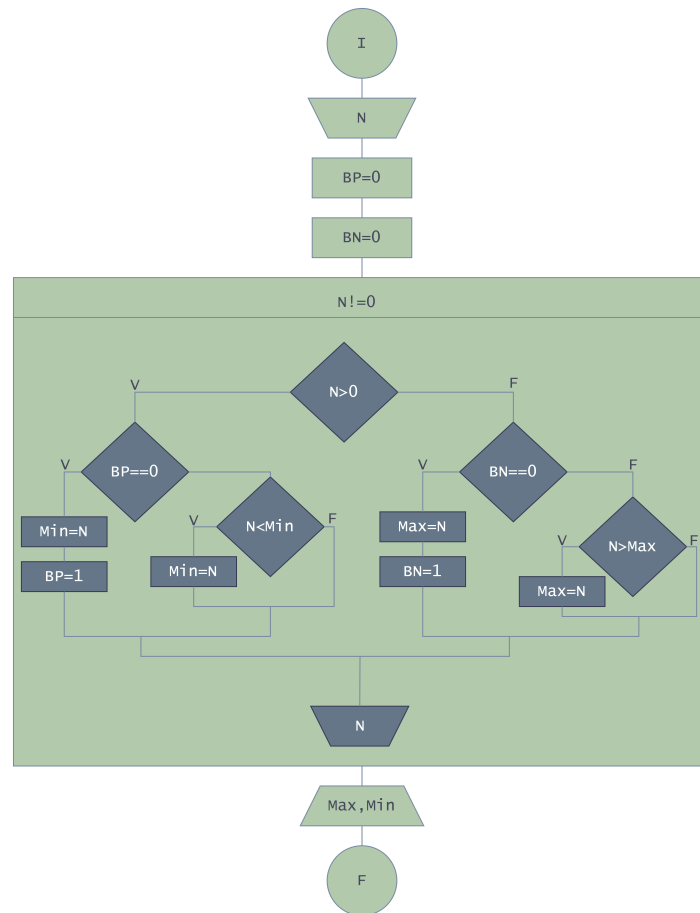
La explicación es la misma que en el ejercicio anterior, vamos a tener en cuenta que el CON va a incrementar en cada vuelta, o sea que nos lleva el conteo, por ende nos sirve para saber en qué momento se ingresaron el M1 y M2.

En el primer IF **B1** la diferencia es que vamos a guardar la posición en que se ingresó el **M1**, esto lo hacemos asignando la posición actual "**CON**" en Posición del primer mínimo **POS1**.

Cuando ingrese al segundo IF **B2** y llegue al condicional IF (**N < M1**) lo mismo, si da verdadero hacemos el enroque de **M1** A **M2** Y DE **N** A **M1**. Lo distinto acá es que también tenemos que hacer el enroque de las posiciones para no perderlas. O sea que la Posición 1 pasa a la posición 2: **POS2 = POS1**; y el conteo actual "**CON**" pasa a la posición 1 **POS1**. En el caso que dé falso simplemente asignamos la posición actual a la posición 2.

Esta explicación del enroque de las posiciones se repite durante todo el programa, lo que hay que tener en claro es que **CON** lleva el "índice" y cada vez que hacemos una asignación tenemos que cuidar el orden para no pisar ningún dato.

10. Hacer un programa que solicite una lista de números que corta cuando se ingresa un cero y luego emitir por pantalla el máximo de los números negativos y el mínimo de los números positivos.



Este programa es como juntar los que ya hicimos y unirlos en uno solo: buscar números positivos y guardar un máximo y un mínimo, ya lo venimos trabajando.

Para empezar, tenemos que tener en cuenta que vamos a discriminar primero en números positivos y negativos y luego de diferenciarlos vamos a trabajar, por un lado, buscando el menor positivo y por el otro, el máximo negativo.

Iniciamos pidiendo el ingreso de un número y seguimos con la creación de 2 banderas inicializadas en 0.

En nuestra condición del ciclo **N != 0**, mientras que el número ingresado sea distinto a 0, sigue iterando.

IF (N > 0): Este IF nos sirve para diferenciar entre números positivos y negativos, si es positivo va a tomar el camino del verdadero, si es negativo va a ir para el camino del falso.

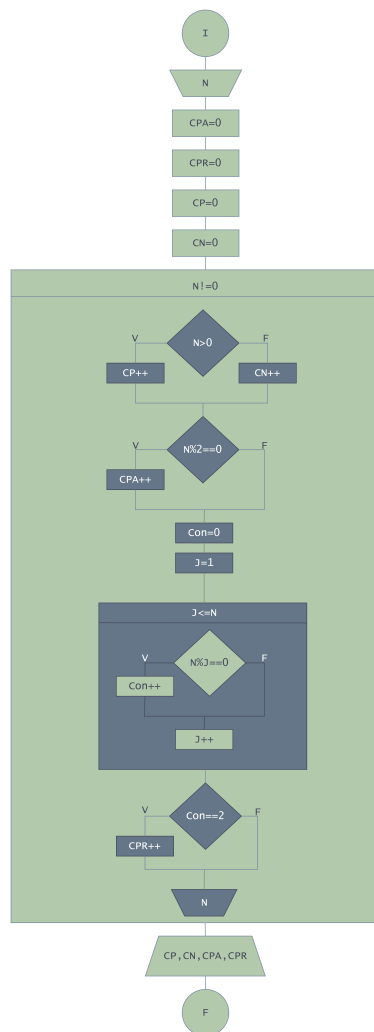
En ambos casos, sea positivo o negativo, se va a topar con el IF que contiene nuestra variable bandera que inicializamos en 0 al inicio del programa "**BP == 0**" y "**BN == 0**". Estos IFs nos van a servir para guardar el primer número positivo que se ingresó y el primer número negativo que se ingresó; en el caso de que sea positivo **MIN = N** y en el caso del negativo **MAX = N**. Luego cambiamos las banderas a 1 para que, cuando se vuelvan a topar con estos IFs, tomen el camino del falso y solo comparen los números ingresados con los que tenemos guardados en nuestras variables en **MIN** para positivos y **MAX** para negativos.

Cuando termine de iterar nuestro ciclo solo nos queda mostrar en pantalla **MIN** y **MAX**.

A modo de resumen para tener en cuenta el funcionamiento de este programa:

1. Diferenciar en positivos y negativos.
2. Utilizar los IF con nuestras banderas para guardar el primer ingreso positivo y negativo.
3. Comparar los números ingresados en cada vuelta con el valor de **MIN** y **MAX**.
4. Con nuestros primeros ingresos ya guardados, el programa va a iterar entre el paso 1 y 3 hasta que se ingrese un 0.

11. Hacer un programa para ingresar una lista de números que corta cuando se ingresa un cero y luego mostrar: la cantidad de números primos, la cantidad de números pares, la cantidad de positivos y la cantidad de negativos.



Para este ejercicio vamos a usar un ciclo dentro de otro. A su vez es pensarlo como unir todos los ejercicios que veníamos haciendo. Buscar números primos, pares, los positivos y negativos.

Empezamos pidiendo el ingreso de un número, luego creamos las variables **CPA** "Cantidad de pares" **CPR** "Cantidad de primos" **CP** "Cantidad de positivos" y **CN** "Cantidad de negativos" todas inicializadas en 0.

En la condición del ciclo como venimos trabajando corta cuando se ingresa un 0: "**N != 0**".

Dentro del ciclo creamos el primer IF (**N > 0**). Este nos sirve para diferenciar entre números positivos y negativos. Si son positivos, va para el verdadero e incrementamos **CP++**. Si da falso es porque fue negativo e incrementamos **CN++**.

En el siguiente IF (**N % 2 == 0**). Este nos sirve para saber si el número ingresado es par, si da verdadero incrementamos el valor de cantidad de pares **CPA++**.

En este punto ya tenemos diferenciado los números positivos, negativos y los pares.

Ahora nos queda saber si este número es primo y para esto creamos la variable contador inicializada en 0 "**con = 0**" y la variable **J** en 1 "**J = 1**". Las creamos en este punto y no afuera del ciclo porque necesitamos que se reinicien sus valores después que itere el ciclo. Es tal cual cuando buscamos un número Primo con anterioridad usando While.

Creamos otro ciclo con la condición **J <= N**. Esto nos dice que mientras **J** sea menor o igual a **N** siga iterando, luego hacemos un IF (**N % J == 0**). Si la división entre **N** y **J** nos da resto 0, vamos al verdadero e incrementamos el contador **CON ++**. Al salir del IF incrementamos el valor de **J**, para que vaya dividiendo con **N** hasta que iguale su valor. Una vez que salió de este ciclo, creamos un IF (**CON == 2**) donde sabremos que si **Con** es igual a 2 es porque el número fue divisible por 1 y por sí mismo, entonces es primo. Aumentamos el contador de Primos **CPR++**.

Luego volvemos a pedir otro número y se va a repetir todo el proceso hasta que se ingrese un 0.

Para finalizar, mostramos en pantalla los valores obtenidos en **CP, CN, CPA, CPR**.

¡Felicidades por completar la Guía 5 con éxito!

Como verás, en esta guía implementamos muchos ejercicios y conocimientos que adquirimos en guías pasadas. Terminamos transformando programas chicos en otros más grandes agrupándolos a todos como, por ejemplo, en el ejercicio 11.

Aprendimos a utilizar el "Ciclo inexacto" While traducción "mientras"; dijimos que va a iterar mientras suceda o no suceda algo. Para que funcione le configuramos una condición que va a ser evaluada para que itere nuestro ciclo.

Tener siempre presente cómo configuramos nuestro ciclo, ya que se torna bastante común que le erremos y se caigamos en un ciclo infinito.

El While es muy similar al Ciclo FOR, una especie de evolución la cual nos sirve para cargar tantos datos como queramos, de ahí lo de "inexacto". Podemos cargar 10, 20, 30 edades por ejemplo, las que necesitemos hasta que lo cortemos.

También vimos que tiene un "primo" más chico: el Do While que si bien no lo utilizamos, en estos ejercicios los podrías incorporar en tus diagramas para jugar un poco.

Diferencias: el Do While lo que hace es que se ejecuta una vez si o si, porque tiene la condición de corte al final del ciclo, a diferencia del While que la tenemos al principio y si no la cumple no se ejecuta ni 1 sola vuelta.

Cuándo usar For o While? Cuando sabemos la cantidad exacta de vueltas que vamos a dar, usamos el FOR; cuando no sabemos cuántas vueltas va a dar nuestro ciclo, usamos un While.

Bueno, hasta acá estamos, nos vemos en la próxima guía, la número 6, donde vamos a ver ciclos anidados y corte de control, temas muy interesantes y divertidos.

Seba.

--

Idea y confección: Sebastián Duarte.

 **@sebateunmate**

Revisión y correcciones: Maximiliano Sar Fernández.

 **@maxiprograma**

--

Esta guía de ejercicios pertenece al "Curso de Programación Desde Cero Nivel Universidad" de Maxi Programa. Cualquier duda o sugerencia, no dejes de consultarla en los foros del aula virtual del curso.

<https://campusvirtualmaxiprograma.com>

<https://maxiprograma.com>